

Assignment 16.5

Name:G.Charanya

Batch:23

HT No:2303A510G6

Task 1 – Employee Management Database

Task:

Create a database schema for managing employee records in an organization.

Instructions:

- Tables: Employees, Departments, Salaries.
- Use AI to define constraints such as NOT NULL and UNIQUE.
- **Generate queries:**
 - o List all employees in a specific department.
 - o Retrieve salary details of an employee.
 - o Find the highest-paid employee in each department.

Expected Output:

- Schema and SQL queries demonstrating joins and constraints.

Code:

```
1  -- Create an employee table
2  CREATE TABLE employee1 (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      first_name VARCHAR(50) NOT NULL,
5      last_name VARCHAR(50) NOT NULL,
6      email VARCHAR(100) NOT NULL UNIQUE,
7      phone_number VARCHAR(20),
8      hire_date DATE NOT NULL,
9      job_title VARCHAR(50) NOT NULL,
10     salary DECIMAL(10, 2) NOT NULL,
11     department_id INT,
12     FOREIGN KEY (department_id) REFERENCES department(id)
13 );
14 -- Create a departments table
15 CREATE TABLE department1 (
16     id INTEGER PRIMARY KEY AUTOINCREMENT,
17     name VARCHAR(100) NOT NULL UNIQUE,
18     manager_id INT,
19     FOREIGN KEY (manager_id) REFERENCES employee1(id)
20 );
21 --Create salaries table
22 CREATE TABLE salary1 (
23     id INTEGER PRIMARY KEY AUTOINCREMENT,
24     employee_id INTEGER NOT NULL,
25     amount DECIMAL(10, 2) NOT NULL,
26     effective_date DATE NOT NULL,
27     FOREIGN KEY (employee_id) REFERENCES employee1(id)
28 );
29 -- Insert sample data into department table
30 INSERT INTO department1 (name, manager_id) VALUES
31 ('Human Resources', NULL),
32 ('Engineering', NULL),
33 ('Marketing', NULL),
34 ('Finance', NULL),
35 ('Sales', NULL);
36 -- Insert sample data into employee table
37 INSERT INTO employee1 (first_name, last_name, email, phone_number, hire_date, job_title, salary, department_id) VALUES
38 ('John', 'Doe', 'john.doe@example.com', '123-456-7890', '2022-01-01', 'Software Engineer', 75000.00, 2),
39 ('Jane', 'Smith', 'jane.smith@example.com', '098-765-4321', '2022-01-01', 'Sales Associate', 50000.00, 3),
40 ('Emily', 'Johnson', 'emily.johnson@example.com', '555-555-5555', '2022-01-01', 'Marketing Specialist', 55000.00, 4),
41 ('Michael', 'Brown', 'michael.brown@example.com', '555-123-4567', '2022-01-01', 'Finance Analyst', 60000.00, 5),
42 ('Sarah', 'Davis', 'sarah.davis@example.com', '555-987-6543', '2022-01-01', 'Human Resources Specialist', 50000.00, 1);
```

```

43 -- Insert sample data into salary table
44 INSERT INTO salary1 (employee_id, amount, effective_date) VALUES
45 (1, 75000.00, '2022-01-01'),
46 (2, 50000.00, '2022-01-01'),
47 (3, 55000.00, '2022-01-01'),
48 (4, 60000.00, '2022-01-01'),
49 (5, 50000.00, '2022-01-01');
50 -- List all employees in a specific department
51 SELECT e.first_name, e.last_name, d.name AS department
52 FROM employee1 e
53 JOIN department1 d ON e.department_id = d.id
54 WHERE d.name = 'Engineering';
55 -- Retrieve the salary details of an employee
56 SELECT e.first_name, e.last_name, s.amount, s.effective_date
57 FROM employee1 e
58 JOIN salary1 s ON e.id = s.employee_id
59 WHERE e.email = 'john.doe@example.com';
60 -- display the highest paid employee in each department
61 SELECT d.name AS department, e.first_name, e.last_name, MAX(s.amount) AS highest_salary
62 FROM employee1 e
63 JOIN department1 d ON e.department_id = d.id
64 JOIN salary1 s ON e.id = s.employee_id
65 GROUP BY d.name;
66

```

Output:

SQL ▾ < 1 / 1 > 1 - 1 of 1			
first_name	last_name	department	
John	Doe	Engineering	

SQL ▾ < 1 / 1 > 1 - 1 of 1			
first_name	last_name	amount	effective_date
John	Doe	75000	2022-01-01

SQL ▾ < 1 / 1 > 1 - 5 of 5			
department	first_name	last_name	highest_salary
Engineering	John	Doe	75000
Finance	Emily	Johnson	55000
Human Resources	Sarah	Davis	50000
Marketing	Jane	Smith	50000
Sales	Michael	Brown	60000

Task 2 – Online Course Platform Database

Task:

Design a database for an online learning platform.

Instructions:

- Tables: Users, Courses, Lessons, Enrollments.
- Define relationships between users and courses.
- Generate queries:
 - o List all courses enrolled by a user.
 - o Count number of learners per course.
 - o Retrieve lesson details for a course.

Expected Output:

- Relational schema with SQL queries using joins.

Code:

```
C: > Users > PC > Downloads > aiac > onlineLearningPlatform.sql
1  -- Create a users table
2  CREATE TABLE users (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      first_name TEXT NOT NULL,
5      last_name TEXT NOT NULL,
6      email TEXT NOT NULL UNIQUE,
7      password TEXT NOT NULL,
8      role TEXT CHECK(role IN ('student', 'instructor')) NOT NULL,
9      registration_date TEXT NOT NULL
10 );
11 -- Create a courses table
12 CREATE TABLE courses (
13     id INTEGER PRIMARY KEY AUTOINCREMENT,
14     title TEXT NOT NULL,
15     description TEXT,
16     instructor_id INT,
17     FOREIGN KEY (instructor_id) REFERENCES users(id)
18 );
19 -- Create a lessons table
20 CREATE TABLE lessons (
21     id INTEGER PRIMARY KEY AUTOINCREMENT,
22     course_id INT,
23     title TEXT NOT NULL,
24     content TEXT,
25     FOREIGN KEY (course_id) REFERENCES courses(id)
26 );
27 -- Create an enrollments table
28 CREATE TABLE enrollments (
29     id INTEGER PRIMARY KEY AUTOINCREMENT,
30     user_id INT,
31     course_id INT,
32     enrollment_date TEXT NOT NULL,
33     FOREIGN KEY (user_id) REFERENCES users(id),
34     FOREIGN KEY (course_id) REFERENCES courses(id)
35 );
36 -- Insert sample data into users table
37 INSERT INTO users (first_name, last_name, email, password, role, registration_date) VALUES
38 ('Alice', 'Johnson', 'alice.johnson@example.com', 'password123', 'student', '2022-01-01'),
39 ('Bob', 'Smith', 'bob.smith@example.com', 'password123', 'student', '2022-01-01'),
40 ('Charlie', 'Brown', 'charlie.brown@example.com', 'password123', 'instructor', '2022-01-01'),
41 ('David', 'Wilson', 'david.wilson@example.com', 'password123', 'student', '2022-01-01');
42 -- Insert sample data into courses table
```

```

42 -- Insert sample data into courses table
43 INSERT INTO courses (title, description, instructor_id) VALUES
44 ('Introduction to Programming', 'Learn the basics of programming using Python.', 3),
45 ('Data Structures and Algorithms', 'Explore fundamental data structures and algorithms.', 3),
46 ('Web Development', 'Build modern web applications using HTML, CSS, and JavaScript.', 3);
47 -- Insert sample data into lessons table
48 INSERT INTO lessons (course_id, title, content) VALUES
49 (1, 'Variables and Data Types', 'In this lesson, you will learn about variables and data types in Python. '),
50 (1, 'Control Structures', 'In this lesson, you will learn about control structures in Python. '),
51 (2, 'Arrays', 'In this lesson, you will learn about arrays in data structures. '),
52 (2, 'Linked Lists', 'In this lesson, you will learn about linked lists in data structures. '),
53 (3, 'HTML Basics', 'In this lesson, you will learn the basics of HTML. '),
54 (3, 'CSS Basics', 'In this lesson, you will learn the basics of CSS. '),
55 (3, 'JavaScript Basics', 'In this lesson, you will learn the basics of JavaScript. ');
56 -- Insert sample data into enrollments table
57 INSERT INTO enrollments (user_id, course_id, enrollment_date) VALUES
58 (1, 1, '2022-01-01'),
59 (1, 2, '2022-01-01'),
60 (2, 1, '2022-01-01'),
61 (3, 1, '2022-01-01'),
62 (4, 3, '2022-01-01');
63 --List all courses enrolled by a user
64 SELECT c.title, e.enrollment_date
65 FROM courses c
66 JOIN enrollments e ON c.id = e.course_id
67 WHERE e.user_id = 1;
68 --Count number of learners enrolled per course
69 SELECT c.title, COUNT(e.user_id) AS enrolled_students
70 FROM courses c
71 JOIN enrollments e ON c.id = e.course_id
72 GROUP BY c.title;
73 -- Retrieve lesson details for a course
74 SELECT l.title, l.content
75 FROM lessons l
76 JOIN courses c ON l.course_id = c.id
77 WHERE c.title = 'Introduction to Programming';

```

Output:

SQL ▾ < 1 / 1 > 1 - 2 of 2 🗑️ ↶ ↷ 🔍

title	enrollment_date
Introduction to Programming	2022-01-01
Data Structures and Algorithms	2022-01-01

SQL ▾ < 1 / 1 > 1 - 3 of 3 🗑️ ↶ ↷ 🔍

title	enrolled_students
Data Structures and Algorithms	1
Introduction to Programming	3
Web Development	1

SQL ▾ < 1 / 1 > 1 - 2 of 2 🗑️ ↶ ↷ 🔍

title	content
Variables and Data Types	In this lesson, you will learn about variables and data types in Python.
Control Structures	In this lesson, you will learn about control structures in Python.

Task 3 – Banking Transaction Database

Task:

Create a schema for a banking transaction management system.

Instructions:

- Tables: Customers, Accounts, Transactions.
- Use AI to suggest constraints for transaction validity.
- **Generate queries:**
 - o Retrieve transaction history for an account.
 - o Calculate total deposits and withdrawals.
 - o Find accounts with balance below a threshold.

Expected Output:

- Secure database schema and analytical SQL queries.

Code:

```
1  -- Create a customers table
2  CREATE TABLE customers (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      first_name VARCHAR(50) NOT NULL,
5      last_name VARCHAR(50) NOT NULL,
6      email VARCHAR(100) NOT NULL UNIQUE,
7      phone_number VARCHAR(20),
8      registration_date DATE NOT NULL
9  );
10 -- Create an accounts table
11 CREATE TABLE accounts (
12     id INTEGER PRIMARY KEY AUTOINCREMENT,
13     customer_id INTEGER NOT NULL,
14     account_type VARCHAR(20) CHECK(account_type IN ('savings', 'checking')) NOT NULL,
15     balance DECIMAL(10, 2) NOT NULL,
16     FOREIGN KEY (customer_id) REFERENCES customers(id)
17 );
18 -- Create a transactions table
19 CREATE TABLE transactions (
20     id INTEGER PRIMARY KEY AUTOINCREMENT,
21     account_id INTEGER NOT NULL,
22     transaction_type VARCHAR(20) CHECK(transaction_type IN ('deposit', 'withdrawal')) NOT NULL,
23     amount DECIMAL(10, 2) NOT NULL,
24     transaction_date DATE NOT NULL,
25     FOREIGN KEY (account_id) REFERENCES accounts(id)
26 );
27 -- Insert sample data into customers table
28 INSERT INTO customers (first_name, last_name, email, phone_number, registration_date) VALUES
29 ('Alice', 'Johnson', 'alice.johnson@example.com', '555-1234', '2022-01-01'),
30 ('Bob', 'Smith', 'bob.smith@example.com', '555-5678', '2022-01-01'),
31 ('Charlie', 'Brown', 'charlie.brown@example.com', '555-9012', '2022-01-01'),
32 ('David', 'Wilson', 'david.wilson@example.com', '555-3456', '2022-01-01');
33 -- Insert sample data into accounts table
34 INSERT INTO accounts (customer_id, account_type, balance) VALUES
35 (1, 'savings', 1000.00),
36 (1, 'checking', 500.00),
37 (2, 'savings', 2000.00),
38 (3, 'checking', 1500.00),
39 (4, 'savings', 3000.00);
40 -- Insert sample data into transactions table
```

```

40 -- Insert sample data into transactions table
41 INSERT INTO transactions (account_id, transaction_type, amount, transaction_date) VALUES
42 (1, 'deposit', 500.00, '2022-01-10'),
43 (1, 'withdrawal', 200.00, '2022-01-15'),
44 (2, 'deposit', 1000.00, '2022-01-12'),
45 (3, 'withdrawal', 300.00, '2022-01-20'),
46 (4, 'deposit', 1500.00, '2022-01-18');
47 -- Retrieve transaction history for an account
48 SELECT t.transaction_type, t.amount, t.transaction_date
49 FROM transactions t
50 JOIN accounts a ON t.account_id = a.id
51 WHERE a.customer_id = 1 AND a.account_type = 'savings';
52 -- Calculate the total deposits and withdrawals
53 SELECT
54     SUM(CASE WHEN transaction_type = 'deposit' THEN amount ELSE 0 END) AS total_deposits,
55     SUM(CASE WHEN transaction_type = 'withdrawal' THEN amount ELSE 0 END) AS total_withdrawals
56 FROM transactions t
57 JOIN accounts a ON t.account_id = a.id
58 WHERE a.customer_id = 1 AND a.account_type = 'savings';
59 -- Find accounts with the balance below a threshold
60 SELECT a.id, a.account_type, a.balance
61 FROM accounts a
62 WHERE a.balance < 1000.00;
63

```

Output:

Output:

SQL ▾ < 1 / 1 > 1 - 2 of 2

transaction_type	amount	transaction_date
deposit	500	2022-01-10
withdrawal	200	2022-01-15

SQL ▾ < 1 / 1 > 1 - 1 of 1

total_deposits	total_withdrawals
500	200

SQL ▾ < 1 / 1 > 1 - 1 of 1

id	account_type	balance
2	checking	500

Task 4 – Airline Reservation Database

Task:

Design a database for an airline reservation system.

Instructions:

- Tables: Flights, Passengers, Tickets, Bookings.
- Define relationships and seat allocation logic.
- **Generate queries:**
 - o List all passengers on a specific flight.
 - o Find available seats for a flight.
 - o Count total bookings per flight.

Expected Output:

- Schema with SQL queries using joins and counts.

Code :

```
1  --Create flight table
2  CREATE TABLE flight (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      flight_number VARCHAR(10) NOT NULL UNIQUE,
5      departure_city VARCHAR(50) NOT NULL,
6      arrival_city VARCHAR(50) NOT NULL,
7      departure_time DATETIME NOT NULL,
8      arrival_time DATETIME NOT NULL,
9      price DECIMAL(10, 2) NOT NULL
10 );
11 --Create passengers table
12 CREATE TABLE passengers (
13     id INTEGER PRIMARY KEY AUTOINCREMENT,
14     first_name VARCHAR(50) NOT NULL,
15     last_name VARCHAR(50) NOT NULL,
16     email VARCHAR(100) NOT NULL UNIQUE,
17     phone_number VARCHAR(20)
18 );
19 --create tickets table
20 CREATE TABLE tickets (
21     id INTEGER PRIMARY KEY AUTOINCREMENT,
22     flight_id INTEGER NOT NULL,
23     passenger_id INTEGER NOT NULL,
24     seat_number VARCHAR(5) NOT NULL,
25     booking_date DATETIME NOT NULL,
26     FOREIGN KEY (flight_id) REFERENCES flight(id),
27     FOREIGN KEY (passenger_id) REFERENCES passengers(id)
28 );
29 --Create bookings table
30 CREATE TABLE bookings (
31     id INTEGER PRIMARY KEY AUTOINCREMENT,
32     passenger_id INTEGER NOT NULL,
33     booking_date DATETIME NOT NULL,
34     total_price DECIMAL(10, 2) NOT NULL,
35     FOREIGN KEY (passenger_id) REFERENCES passengers(id)
36 );
37 --Insert sample data into flight table
38 INSERT INTO flight (flight_number, departure_city, arrival_city, departure_time, arrival_time, price) VALUES
39 ('AA101', 'New York', 'Los Angeles', '2022-01-01 08:00:00', '2022-01-01 11:00:00', 300.00),
40 ('BA202', 'London', 'Paris', '2022-01-02 09:00:00', '2022-01-02 10:30:00', 150.00),
41 ('CA303', 'Beijing', 'Shanghai', '2022-01-03 10:00:00', '2022-01-03 12:00:00', 200.00);
```

```

42 --Insert sample data into passengers table
43 INSERT INTO passengers (first_name, last_name, email, phone_number) VALUES
44 ('Alice', 'Johnson', 'alice.johnson@example.com', '555-123-4567'),
45 ('Bob', 'Smith', 'bob.smith@example.com', '555-987-6543'),
46 ('Charlie', 'Brown', 'charlie.brown@example.com', '555-555-5555');
47 --Insert sample data into tickets table
48 INSERT INTO tickets (flight_id, passenger_id, seat_number, booking_date) VALUES
49 (1, 1, '12A', '2022-01-01 09:00:00'),
50 (1, 2, '12B', '2022-01-01 09:30:00'),
51 (2, 3, '5C', '2022-01-02 10:00:00');
52 --Insert sample data into bookings table
53 INSERT INTO bookings (passenger_id, booking_date, total_price) VALUES
54 (1, '2022-01-01 09:00:00', 300.00),
55 (2, '2022-01-01 09:30:00', 300.00),
56 (3, '2022-01-02 10:00:00', 150.00);
57 --List all passengers on a specific flight
58 SELECT p.first_name, p.last_name, t.seat_number
59 FROM passengers p
60 JOIN tickets t ON p.id = t.passenger_id
61 WHERE t.flight_id = 1;
62 --Find available seats for a flight
63 SELECT f.seat_number
64 FROM tickets f
65 WHERE f.flight_id = 1
66 AND f.seat_number NOT IN (
67     SELECT t.seat_number
68     FROM tickets t
69     WHERE t.flight_id = 1
70 );
71 -- Count total bookings per flight
72 SELECT f.flight_number, COUNT(t.id) AS total_bookings
73 FROM flight f
74 LEFT JOIN tickets t ON f.id = t.flight_id
75 GROUP BY f.flight_number;

```

Output:

first_name	last_name	seat_number
Alice	Johnson	12A
Bob	Smith	12B

flight_number	total_bookings
AA101	2
BA202	1
CA303	0

Task 5 – Transport Management Database

Task:

Design a database for transport route management.

Instructions:

- Tables: Vehicles, Routes, Drivers, Assignments.
- Use AI to suggest indexing strategies.
- **Generate queries:**
 - o List vehicles assigned to a route.
 - o Retrieve driver details for a vehicle.
 - o Count trips per route.

Expected Output:

- Schema and optimized SQL queries.

Code:

```
1  --Create vehicles table
2  CREATE TABLE vehicles (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      make VARCHAR(50) NOT NULL,
5      model VARCHAR(50) NOT NULL,
6      year INTEGER NOT NULL,
7      license_plate VARCHAR(20) NOT NULL UNIQUE
8  );
9  --create routes table
10 CREATE TABLE routes (
11     id INTEGER PRIMARY KEY AUTOINCREMENT,
12     origin VARCHAR(50) NOT NULL,
13     destination VARCHAR(50) NOT NULL,
14     distance DECIMAL(10, 2) NOT NULL
15 );
16 --create drivers table
17 CREATE TABLE drivers (
18     id INTEGER PRIMARY KEY AUTOINCREMENT,
19     first_name VARCHAR(50) NOT NULL,
20     last_name VARCHAR(50) NOT NULL,
21     license_number VARCHAR(20) NOT NULL UNIQUE,
22     phone_number VARCHAR(20)
23 );
24 --create assignments table
25 CREATE TABLE assignments (
26     id INTEGER PRIMARY KEY AUTOINCREMENT,
27     vehicle_id INTEGER NOT NULL,
28     driver_id INTEGER NOT NULL,
29     route_id INTEGER NOT NULL,
30     assignment_date DATE NOT NULL,
31     FOREIGN KEY (vehicle_id) REFERENCES vehicles(id),
32     FOREIGN KEY (driver_id) REFERENCES drivers(id),
33     FOREIGN KEY (route_id) REFERENCES routes(id)
34 );
35 --Insert sample data into vehicles table
36 INSERT INTO vehicles (make, model, year, license_plate) VALUES
37 ('Toyota', 'Camry', 2020, 'ABC123'),
38 ('Honda', 'Civic', 2019, 'XYZ789'),
39 ('Ford', 'F-150', 2021, 'DEF456');
40 --Insert sample data into routes table
41 INSERT INTO routes (origin, destination, distance) VALUES
42 ('New York', 'Los Angeles', 2800.00),
43 ('Chicago', 'Houston', 1080.00),
44 ('Miami', 'Atlanta', 660.00);
```

```

45 --Insert sample data into drivers table
46 INSERT INTO drivers (first_name, last_name, license_number, phone_number) VALUES
47 ('John', 'Doe', 'D1234567', '555-123-4567'),
48 ('Jane', 'Smith', 'D2345678', '555-987-6543'),
49 ('Emily', 'Johnson', 'D3456789', '555-555-5555');
50 --Insert sample data into assignments table
51 INSERT INTO assignments (vehicle_id, driver_id, route_id, assignment_date) VALUES
52 (1, 1, 1, '2022-01-01'),
53 (2, 2, 2, '2022-01-02'),
54 (3, 3, 3, '2022-01-03');
55 --List vehicles assigned to a route.
56 SELECT v.make, v.model, v.year, v.license_plate, r.origin, r.destination
57 FROM vehicles v
58 JOIN assignments a ON v.id = a.vehicle_id
59 JOIN routes r ON a.route_id = r.id
60 WHERE r.id = 1;
61 --Retrieve driver details for a vehicle
62 SELECT d.first_name, d.last_name, d.license_number, d.phone_number
63 FROM drivers d
64 JOIN assignments a ON d.id = a.driver_id
65 WHERE a.vehicle_id = 1;
66 --Count trips per route
67 SELECT r.id, r.origin, r.destination, COUNT(a.id) AS trip_count
68 FROM routes r
69 LEFT JOIN assignments a ON r.id = a.route_id
70 GROUP BY r.id, r.origin, r.destination;

```

Output:

SQL ▾

< 1 / 1 > 1 - 1 of 1

make	model	year	license_plate	origin	destination
Toyota	Camry	2020	ABC123	New York	Los Angeles

SQL ▾

< 1 / 1 > 1 - 1 of 1

first_name	last_name	license_number	phone_number
John	Doe	D1234567	555-123-4567

SQL ▾

< 1 / 1 > 1 - 3 of 3

id	origin	destination	trip_count
1	New York	Los Angeles	1
2	Chicago	Houston	1
3	Miami	Atlanta	1

Task 6 – Hotel Reservation Database

Task:

Create a database schema for hotel reservations.

Instructions:

- Tables: Hotels, Rooms, Guests, Reservations.
- Define date constraints and relationships.
- **Generate queries:**
 - o List all reservations for a guest.
 - o Find available rooms for a date range.
 - o Calculate occupancy rate per hotel.

Expected Output:

- Relational schema and SQL queries for booking analysis.

Code:

```
1  --create hotels table
2  CREATE TABLE hotels (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      name VARCHAR(100) NOT NULL,
5      address VARCHAR(200) NOT NULL,
6      city VARCHAR(50) NOT NULL,
7      state VARCHAR(50) NOT NULL,
8      zip_code VARCHAR(10) NOT NULL
9  );
10 --create rooms table
11 CREATE TABLE rooms (
12     id INTEGER PRIMARY KEY AUTOINCREMENT,
13     hotel_id INTEGER NOT NULL,
14     room_number VARCHAR(10) NOT NULL,
15     room_type VARCHAR(20) CHECK(room_type IN ('single', 'double', 'suite')) NOT NULL,
16     price DECIMAL(10, 2) NOT NULL,
17     FOREIGN KEY (hotel_id) REFERENCES hotels(id)
18 );
19 --create guests table
20 CREATE TABLE guests (
21     id INTEGER PRIMARY KEY AUTOINCREMENT,
22     first_name VARCHAR(50) NOT NULL,
23     last_name VARCHAR(50) NOT NULL,
24     email VARCHAR(100) NOT NULL UNIQUE,
25     phone_number VARCHAR(20)
26 );
27 --create reservations table
28 CREATE TABLE reservations (
29     id INTEGER PRIMARY KEY AUTOINCREMENT,
30     guest_id INTEGER NOT NULL,
31     room_id INTEGER NOT NULL,
32     check_in_date DATE NOT NULL,
33     check_out_date DATE NOT NULL,
34     total_price DECIMAL(10, 2) NOT NULL,
35     FOREIGN KEY (guest_id) REFERENCES guests(id),
36     FOREIGN KEY (room_id) REFERENCES rooms(id)
37 );
38 --Insert sample data into hotels table
39 INSERT INTO hotels (name, address, city, state, zip_code) VALUES
40 ('Grand Hotel', '123 Main St', 'New York', 'NY', '10001'),
41 ('Ocean View Resort', '456 Beach Ave', 'Miami', 'FL', '33101'),
42 ('Mountain Lodge', '789 Alpine Rd', 'Denver', 'CO', '80201');
```

```

43 --Insert sample data into rooms table
44 INSERT INTO rooms (hotel_id, room_number, room_type, price) VALUES
45 (1, '101', 'single', 100.00),
46 (1, '102', 'double', 150.00),
47 (2, '201', 'suite', 300.00),
48 (2, '202', 'double', 200.00),
49 (3, '301', 'single', 120.00);
50 --Insert sample data into guests table
51 INSERT INTO guests (first_name, last_name, email, phone_number) VALUES
52 ('Alice', 'Johnson', 'alice.johnson@example.com', '555-123-4567'),
53 ('Bob', 'Smith', 'bob.smith@example.com', '555-987-6543'),
54 ('Charlie', 'Brown', 'charlie.brown@example.com', '555-555-5555');
55 --Insert sample data into reservations table
56 INSERT INTO reservations (guest_id, room_id, check_in_date, check_out_date, total_price) VALUES
57 (1, 1, '2022-01-01', '2022-01-05', 400.00),
58 (2, 3, '2022-01-10', '2022-01-15', 1500.00),
59 (3, 5, '2022-01-20', '2022-01-25', 600.00);
60 --List all reservations for a guest
61 SELECT * FROM reservations WHERE guest_id = 1;
62 --Find available rooms for a date range
63 SELECT * FROM rooms WHERE id NOT IN (
64     SELECT room_id FROM reservations
65     WHERE check_in_date <= '2022-01-10' AND check_out_date >= '2022-01-05'
66 );
67 --Calculate occupancy rate per hotel
68 SELECT h.name, COUNT(r.id) AS occupied_rooms, COUNT(ro.id) AS total_rooms,
69        (COUNT(r.id) * 100.0 / COUNT(ro.id)) AS occupancy_rate
70 FROM hotels h
71 LEFT JOIN rooms ro ON h.id = ro.hotel_id
72 LEFT JOIN reservations r ON ro.id = r.room_id AND r.check_in_date <= '2022-01-10' AND r.check_out_date >= '2022-01-05'
73 GROUP BY h.id, h.name;

```

Output:

SQL ▼ < 1 / 1 > 1 - 1 of 1

id	guest_id	room_id	check_in_date	check_out_date	total_price
1	1	1	2022-01-01	2022-01-05	400

SQL ▼ < 1 / 1 > 1 - 3 of 3

id	hotel_id	room_number	room_type	price
2	1	102	double	150
4	2	202	double	200
5	3	301	single	120

SQL ▼ < 1 / 1 > 1 - 3 of 3

name	occupied_rooms	total_rooms	occupancy_rate
Grand Hotel	1	2	50.0
Ocean View Resort	1	2	50.0
Mountain Lodge	0	1	0.0